

experiment. The specific code you need will depend on your app, but here are the general steps. For a complete code example, see [Create and run your first experiment with A/B testing](#).

1. In code that runs when the user reaches an objective for one of the goals of the experiment, call the [LogForVariation](#) method again and pass the [StoreServicesExperimentVariation](#) object and the name of a conversion event for your experiment. This must match one of the conversion event names that you enter for your experiment in Partner Center.

The following example logs a conversion event named **userClickedButton** from the **Click** event handler for a button. In this example, the goal of the experiment is to get the user to click the button.

```
C#

private void button_Click(object sender, RoutedEventArgs e)
{
    if (logger == null)
    {
        logger = StoreServicesCustomEventLogger.GetDefault();
    }

    logger.LogForVariation(variation, "userClickedButton");
}
```

Next steps

After you code the experiment in your app, you are ready for the following steps:

1. [Define your experiment in Partner Center](#). Create an experiment that defines the view events, conversion events, and unique variations for your A/B test.
2. [Run and manage your experiment in Partner Center](#).

Related topics

- [Create a project and define remote variables in Partner Center](#)
- [Define your experiment in Partner Center](#)
- [Manage your experiment in Partner Center](#)
- [Create and run your first experiment with A/B testing](#)
- [Run app experiments with A/B testing](#)

Define your experiment in Partner Center

Article • 01/04/2023

After you [create a project and define remote variables in Partner Center](#) and [code your app for experimentation](#), you are ready to create an experiment in the project. When you create the experiment, you will define the goals and variations that your users will receive.

For a walkthrough that demonstrates the end-to-end process of creating and running an experiment, see [Create and run your first experiment with A/B testing](#).

Create your experiment

1. Sign in to [Partner Center](#).
2. Under **Your apps**, select the app for which you want to create an experiment.
3. In the navigation pane, select **Services** and then select **Experimentation**.
4. On the **Experimentation** page, identify the project where you want to add an experiment in the projects table, and click the **Add experiment** link for that project.
5. In the **Experiment name** field, type a name that you can use to easily identify the experiment. After you create an experiment, this name appears in the list of existing experiments on the **Experimentation** page for your app and on the project's page.
6. If you want to edit the experiment while it is active, click the **Editable experiment** check box. Check this box only if you are creating an experiment to validate all the variations through internal testing. For more information, see [Create an experiment for internal testing](#).

ⓘ Note

Do not check this box if you are creating an experiment that you will release to customers (that is, an experiment that is associated with a project ID that is used in a version of your app that is available to customers). Editing an experiment while it is active will invalidate the experiment results.

7. In the **Project name** drop-down, the current project is automatically selected. If you want to add the new experiment to a different project, you can select that project here. Otherwise, leave this selection alone.
8. Make note of the **Project ID** value. When you [code your app for experimentation](#), you must reference this ID in your code so you can receive variation data and report view and conversion events to Partner Center.
9. In the **View event** section, type the name of the [view event](#) for your experiment in the **View event name** field.
10. In the **Goals and conversion events** section, define at least one goal for your experiment:
 - In the **Goal name** field, type a descriptive name for your goal. After you run an experiment, this name appears in the results summary for the experiment.
 - In the **Conversion event name** field, type the name of the [conversion event](#) for this goal.
 - In the **Objective** field, choose **Maximize** or **Minimize**, depending on whether you want to maximize or minimize the occurrences of the conversion event. This information is used in the results summary for the experiment.

Note

Partner Center reports only the first conversion event for each user view in a 24-hour time period. If a user triggers multiple conversion events in your app within a 24-hour period, only the first conversion event is reported. This is intended to help prevent a single user from skewing the experiment results for a sample group of users when the goal is to maximize the number of users who perform a conversion.

Define the remote variables and variations for your experiment

Next, define the remote [variables](#) and [variations](#) for your experiment.

1. In the **Remote variables and variations** section, you should see two default variations, **Variation A (Control)** and **Variation B**. If you want more variations, click **Add variation**. Optionally, you can rename each variation.
2. By default, variations are distributed equally to your app users. If you want to choose a specific distribution percentage, clear the **Distribute equally** check box and type the percentages in the **Distribution (%)** row.

3. Add remote variables to your variations. In the drop-down control at the bottom of this section, choose each variable you want to add and click **Add variable**.

ⓘ **Note**

The variables listed in this control are inherited from the project for the experiment. The default value for the variable (as defined in the project) is automatically assigned to the control variation. If you want to create new variables that aren't listed here, go to the related project page and add the variables there.

4. Edit the variable values for each unique variation in the experiment (that is, the variations other than the control variation).

Save and activate your experiment

When you finish entering the required fields for your experiment, click **Save** to save your experiment.

If you are satisfied with the parameters of your experiment and you are ready to activate it so you can start collecting experiment data from your app, click **Activate**. When the experiment is active, your app can retrieve variation variables and report view and conversion events to Partner Center. For more information, see [Run and manage your experiment in Partner Center](#).

ⓘ **Important**

A project can only contain one active experiment at a time. After you activate an experiment, you can no longer modify the experiment parameters unless you selected the **Editable experiment** check box when you created the experiment. We recommend that you code the experiment in your app before activating your experiment.

Create an experiment for internal testing

You might want to test your experiment with a controlled audience (for example, a set of internal testers) and confirm that all of the variations are working as expected before you activate the experiment for your customers. You can accomplish this by creating an experiment that has the **Editable experiment** option selected.

To test your experiment before releasing it to customers, follow this process:

1. Create two projects: one for the public build of your app, and one for a private build of your app that is available only to your test audience. The following instructions refer to these projects as the public project and test project, respectively.
2. When you [code your app for experimentation](#), reference the project ID from your public project in the public build of your app. In the private build of your app, reference the project ID from your test project.
3. Create an experiment in the test project, and select the **Editable experiment** option for the experiment.
4. Activate the experiment in the test project. Allocate 100% distribution to one variation and verify that this variation works as expected for your testers. Repeat the process for other variations.
5. After you verify that the variations are working as expected, make any final changes to the experiment in the test project. When you are ready to release the experiment to your customers, clone the experiment to the public project. In this experiment, do not select the **Editable experiment** option.
6. Ensure that the target variation distribution is correct in the cloned experiment.
7. Activate the cloned experiment to release the experiment to your customers.

Next steps

After you define your experiment in Partner Center and code the experiment in your app, you are ready to [run and manage your experiment in Partner Center](#).

Related topics

- [Create a project and define remote variables in Partner Center](#)
- [Code your app for experimentation](#)
- [Manage your experiment in Partner Center](#)
- [Create and run your first experiment with A/B testing](#)
- [Run app experiments with A/B testing](#)

Manage your experiment in Partner Center

Article • 10/20/2022

After you [define your experiment in Partner Center](#) and [code your app for experimentation](#), you are ready to activate your experiment and use Partner Center to review the results of your experiment. After you have obtained all the data you need, you can end your experiment and choose whether to keep using the variable values in the control variation for all your apps, or switch to using the variable values in one of your other variations.

ⓘ Note

When you activate an experiment, Partner Center immediately starts collecting data from any apps that are instrumented to log data for your experiment. However, it can take several hours for experiment data to appear in Partner Center.

For a walkthrough that demonstrates the end-to-end process of creating and running an experiment, see [Create and run your first experiment with A/B testing](#).

Activate your experiment

When you are satisfied with the parameters of your experiment in Partner Center and you have updated your app code, you are ready to activate your experiment so you can start collecting experiment data from your app. When the experiment is active, your app can retrieve variation values and report view and conversion events to Partner Center.

1. Sign in to [Partner Center](#).
2. Under **Your apps**, select the app with the experiment that you want to activate.
3. In the navigation pane, select **Services** and then select **Experimentation**.
4. In the table of projects in the **Projects** section, expand the project that contains your experiment and then do one of the following:
 - Click the **Activate** link for your experiment. Your experiment is added to the **Active experiments** section near the top of the page.
 - Click the experiment name, scroll to the bottom of the experiment page, and click **Activate**.

ⓘ Important

After you activate an experiment, you can no longer modify the experiment parameters unless you clicked the **Editable experiment** check box when you created the experiment. We recommend that you code the experiment in your app before activating your experiment.

Review the results of your experiment

1. In Partner Center, return to the **Experimentation** page for your app.
2. In the **Active experiments** section, click the name of your active experiment to go to the experiment page.
3. For an active or completed experiment, the first two sections in this page provide the results of your experiment:
 - The **Results summary** section lists your experiment goals and the conversion rate percentage for each variation.
 - The **Results details** section provides more details for each variation of all the goals in your experiment, including the views, conversions, unique users, conversion rate, delta %, confidence, and significance. The *confidence* is a statistical measure of the reliability of an estimate, which calculates the margin of error. The *significance* is a statistical measure, based on sample size, to determine the likelihood that a result is not due to chance, but is instead attributed to a specific cause.

Note

Partner Center reports only the first conversion event for each user in a 24-hour time period. If a user triggers multiple conversion events in your app within a 24-hour period, only the first conversion event is reported. This is intended to help prevent a single user with many conversion events from skewing the experiment results for a sample group of users.

Complete your experiment

1. In Partner Center, return to your experiment page. For instructions, see the previous section.
2. In the **Results summary** section, do one of the following:
 - If you want to end the experiment and continue using the variable values in the control variation in your app, click **Keep**.

- If you want to end the experiment but switch to using the variable values in a different variation in your app, click **Switch** under the variation to which you want to switch.
3. Click **OK** to confirm that you want to end the experiment.

Related topics

- [Create a project and define remote variables in Partner Center](#)
- [Code your app for experimentation](#)
- [Define your experiment in Partner Center](#)
- [Create and run your first experiment with A/B testing](#)
- [Run app experiments with A/B testing](#)

Create and run your first experiment

Article • 10/24/2022

In this walkthrough, you will:

- Create an experimentation [project](#) in Partner Center that defines several remote variables that represent the text and color of an app button.
- Create an app with code that retrieves the remote variable values, uses this data to change the background color of a button, and logs view and conversion event data back to Partner Center.
- Create an experiment in the project to test whether changing the background color of the app button successfully increases the number of button clicks.
- Run the app to gather experiment data.
- Review the experiment results in Partner Center, choose a variation to enable for all users of the app, and complete the experiment.

For an overview of A/B testing with Partner Center, see [Run app experiments with A/B testing](#).

Prerequisites

To follow this walkthrough, you must have a Partner Center account and you must configure your development computer as described in [Run app experiments with A/B testing](#).

Create a project with remote variables in Partner Center

1. Sign in to [Partner Center](#).
2. If you already have an app in Partner Center that you want to use to create an experiment, select that app in Partner Center. If you do not yet have an app in Partner Center, [create a new app by reserving a name](#) and then select that app in Partner Center.
3. In the navigation pane, click **Services** and then click **Experimentation**.
4. In the **Projects** section of the next page, click the **New project** button.
5. In the **New project** page, enter the name **Button Click Experiments** for your new project.
6. Expand the **Remote variables** section and click **Add variable** four times. You should now have four empty variable rows.

- In the first row, type **buttonText** for the variable name and type **Grey Button** in the **Default value** column.
 - In the second row, type **r** for the variable name and type **128** in the **Default value** column.
 - In the third row, type **g** for the variable name and type **128** in the **Default value** column.
 - In the fourth row, type **b** for the variable name and type **128** in the **Default value** column.
7. Click **Save** and make note of the **project ID** value that appears in the **SDK integration** section. In the next section, you will update your app code and reference this value in your code.

Code the experiment in your app

1. In Visual Studio, create a new Universal Windows Platform project using Visual C#. Name the project **SampleExperiment**.
2. In Solution Explorer, expand your project node, right-click **References**, and click **Add Reference**.
3. In **Reference Manager**, expand **Universal Windows** and click **Extensions**.
4. In the list of SDKs, select the check box next to **Microsoft Engagement Framework** and click **OK**.
5. In **Solution Explorer**, double-click **MainPage.xaml** to open the designer for the main page in the app.
6. Drag a **Button** from **Toolbox** to the page.
7. Double-click the button on the designer to open the code file and add an event handler for the **Click** event.
8. Replace the entire contents of the code file with the following code. Assign the `projectId` variable to the **project ID** value that you obtained from Partner Center in the previous section.

C#

```
using System;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Media;
using System.Threading.Tasks;
```

```

using Windows.UI;
using Windows.UI.Core;

// Namespace for A/B testing.
using Microsoft.Services.Store.Engagement;

namespace SampleExperiment
{
    public sealed partial class MainPage : Page
    {
        private StoreServicesExperimentVariation variation;
        private StoreServicesCustomEventLogger logger;

        // Assign this variable to the project ID for your experiment
        from Dev Center.
        private string projectId = "";

        public MainPage()
        {
            this.InitializeComponent();

            // Because this call is not awaited, execution of the
            current method
            // continues before the call is completed.
#pragma warning disable CS4014
            InitializeExperiment();
#pragma warning restore CS4014
        }

        private async Task InitializeExperiment()
        {
            // Get the current cached variation assignment for the
            experiment.
            var result = await
StoreServicesExperimentVariation.GetCachedVariationAsync(projectId);
            variation = result.ExperimentVariation;

            // Check whether the cached variation assignment needs to
            be refreshed.
            // If so, then refresh it.
            if (result.ErrorCode !=
StoreServicesEngagementErrorCode.None ||
result.ExperimentVariation.IsStale)
            {
                result = await
StoreServicesExperimentVariation.GetRefreshedVariationAsync(projectId);

                // If the call succeeds, use the new result. Otherwise,
                use the cached value.
                if (result.ErrorCode ==
StoreServicesEngagementErrorCode.None)
                {
                    variation = result.ExperimentVariation;
                }
            }
        }
    }
}

```

```

        // Get remote variables named "buttonText", "r", "g", and
        "b" from the variation
        // assignment. If no variation assignment is available, the
        variables default
        // to "Grey button" for the button text and grey RGB value
        for the button color.
        var buttonText = variation.GetString("buttonText", "Grey
Button");
        var r = (byte)variation.GetInt32("r", 128);
        var g = (byte)variation.GetInt32("g", 128);
        var b = (byte)variation.GetInt32("b", 128);

        // Assign button text and color.
        await button.Dispatcher.RunAsync(
            CoreDispatcherPriority.Normal,
            () =>
            {
                button.Background = new
SolidColorBrush(Color.FromArgb(255, r, g, b));
                button.Content = buttonText;
                button.Visibility = Visibility.Visible;
            });

        // Log the view event named "userViewedButton" to Dev
Center.
        if (logger == null)
        {
            logger = StoreServicesCustomEventLogger.GetDefault();
        }

        logger.LogForVariation(variation, "userViewedButton");
    }

    private void button_Click(object sender, RoutedEventArgs e)
    {
        // Log the conversion event named "userClickedButton" to
Dev Center.
        if (logger == null)
        {
            logger = StoreServicesCustomEventLogger.GetDefault();
        }

        logger.LogForVariation(variation, "userClickedButton");
    }
}
}

```

9. Save the code file and build the project.

Create the experiment in Partner Center

1. Return to the **Button Click Experiments** project page in Partner Center.
2. In the **Experiments** section, click the **New experiment** button.
3. In the **Experiment details** section, type the name **Optimize Button Clicks** in the **Experiment name** field.
4. In the **View event** section, type **userViewedButton** in the **View event name** field.
Note that this name matches the view event string that you logged in the code you added in the previous section.
5. In the **Goals and conversion events** section, enter the following values:
 - In the **Goal name** field, type **Increase Button Clicks**.
 - In the **Conversion event name** field, type the name **userClickedButton**. Note that this name matches the conversion event string that you logged in the code you added in the previous section.
 - In the **Objective** field, choose **Maximize**.
6. In the **Remote variables and variations** section, confirm that the **Distribute equally** check box is selected so that the variations will be distributed equally to your app.
7. Add variables to your experiment:
 - a. Click the drop-down control, choose **buttonText**, and click **Add variable**. The string **Grey Button** should automatically appear in the **Variation A** column (this value is derived from the project settings). In the **Variation B** column, type **Blue Button**.
 - b. Click the drop-down control again, choose **r**, and click **Add variable**. The string **128** should automatically appear in the **Variation A** column. In the **Variation B** column, type **1**.
 - c. Click the drop-down control again, choose **g**, and click **Add variable**. The string **128** should automatically appear in the **Variation A** column. In the **Variation B** column, type **1**.
 - d. Click the drop-down control again, choose **b**, and click **Add variable**. The string **128** should automatically appear in the **Variation A** column. In the **Variation B** column, type **255**.
8. Click **Save** and then click **Activate**.

Important

After you activate an experiment, you can no longer modify the experiment parameters unless it you clicked the **Editable experiment** check box when you created the experiment. Typically, we recommend that you code the experiment in your app before activating your experiment.

Run the app to gather experiment data

1. Run the **SampleExperiment** app you created earlier.
2. Confirm that you see either a grey or blue button. Click the button and then close the app.
3. Repeat the above steps several times on the same computer to confirm that your app shows the same button color.

Review the results and complete the experiment

Wait at least several hours after completing the previous section, and then follow these steps to review the results of your experiment and complete the experiment.

ⓘ Note

As soon as you activate an experiment, Partner Center immediately starts collecting data from any apps that are instrumented to log data for your experiment. However, it can take several hours for experiment data to appear in Partner Center.

1. In Partner Center, return to the **Experimentation** page for your app.
2. In the **Active experiments** section, click **Optimize Button Clicks** to go to the page for this experiment.
3. Confirm that the results shown in the **Results summary** and **Results details** sections matches what you expect to see. For more details about these sections, see [Manage your experiment in Partner Center](#).

ⓘ Note

Partner Center reports only the first conversion event for each user in a 24-hour time period. If a user triggers multiple conversion events in your app within a 24-hour period, only the first conversion event is reported. This is intended to help prevent a single user with many conversion events from skewing the experiment results for a sample group of users.

4. Now you are ready to end the experiment. In the **Results summary** section, in the **Variation B** column, click **Switch**. This switches all users of your app to the blue button.

5. Click **OK** to confirm that you want to end the experiment.
6. Run the **SampleExperiment** app you created in the previous section.
7. Confirm that you see a blue button. Note that it may take up to two minutes for your app to receive an updated variation assignment.

Related topics

- [Create a project and define remote variables in Partner Center](#)
- [Code your app for experimentation](#)
- [Define your experiment in Partner Center](#)
- [Manage your experiment in Partner Center](#)
- [Run app experiments with A/B testing](#)

Launch Feedback Hub from your app

Article • 11/01/2022

You can encourage your customers to leave feedback by adding a control (such as a button) to your Universal Windows Platform (UWP) app that launches Feedback Hub. Feedback Hub is a preinstalled app that provides a single place to gather feedback on Windows and installed apps. All customer feedback that is submitted for your app through Feedback Hub is collected and presented to you in the [Feedback report](#) in Partner Center, so you can see the problems, suggestions, and upvotes that your customers have submitted in one report.

To launch Feedback Hub from your app, use an API that is provided by the [Microsoft Store Services SDK](#). We recommend that you use this API to launch Feedback Hub from a UI element in your app that follows our design guidelines.

ⓘ Note

Feedback Hub is available only on devices that run version 10.0.14271 or later of a Windows that is based on the desktop and mobile **device families**. We recommend that you show a feedback control in your app only if the Feedback Hub is available on the user's device. The code in this topic demonstrates how to do this.

How to launch Feedback Hub from your app

To launch Feedback Hub from your app:

1. [Install the Microsoft Store Services SDK](#).
2. Open your project in Visual Studio.
3. In Solution Explorer, right-click the **References** node for your project and click **Add Reference**.
4. In **Reference Manager**, expand **Universal Windows** and click **Extensions**.
5. In the list of SDKs, click the check box next to **Microsoft Engagement Framework** and click **OK**.
6. In your project, add the control that you want to show to users to launch Feedback Hub, such as a button. We recommend that you configure the control as follows:
 - Set the font of the content shown in the control to **Segoe MDL2 Assets**.
 - Set the text in the control to the hexadecimal Unicode character code E939. This is the character code for the recommended feedback icon in the **Segoe MDL2 Assets**

font.

- Set the visibility of the control to hidden.

ⓘ Note

We recommend that you hide your feedback control by default and show it in your initialization code only if the Feedback Hub is available on the user's device. The next step demonstrates how to do this.

The following code demonstrates the XAML definition of a [Button](#) that is configured as described above.

XML

```
<Button x:Name="feedbackButton" FontFamily="Segoe MDL2 Assets"
Content="ⓘ" HorizontalAlignment="Left" Margin="138,352,0,0"
VerticalAlignment="Top" Visibility="Collapsed"
Click="feedbackButton_Click"/>
```

7. In your initialization code for the app page that hosts your feedback control, use the static [IsSupported](#) method of the [StoreServicesFeedbackLauncher](#) class to determine whether the Feedback Hub is available on the user's device. Feedback Hub is available only on devices that run version 10.0.14271 or later of a Windows 10 OS that is based on the desktop and mobile [device families](#).

If this property returns **true**, make the control visible. The following code demonstrates how to do this for a [Button](#).

C#

```
if
(Microsoft.Services.Store.Engagement.StoreServicesFeedbackLauncher.IsSupported())
{
    this.feedbackButton.Visibility = Visibility.Visible;
}
```

ⓘ Note

Although Feedback Hub is not supported on Xbox devices at this time, the **IsSupported** property currently returns **true** on Xbox devices that run version 10.0.14271 or later of Windows. This is a known issue that will be fixed in a future release of the Microsoft Store Services SDK.

8. In the event handler that runs when the user clicks the control, get a [StoreServicesFeedbackLauncher](#) object and call the [LaunchAsync](#) method to launch the Feedback Hub app. There are two overloads for this method: one without parameters, and another one that accepts a dictionary of key and value pairs that contain metadata that you want to associate with the feedback. The following example demonstrates how to launch Feedback Hub in the [Click](#) event handler for a [Button](#).

```
C#  
  
private async void feedbackButton_Click(object sender, RoutedEventArgs  
e)  
{  
    var launcher =  
Microsoft.Services.Store.Engagement.StoreServicesFeedbackLauncher.GetDe  
fault();  
    await launcher.LaunchAsync();  
}
```

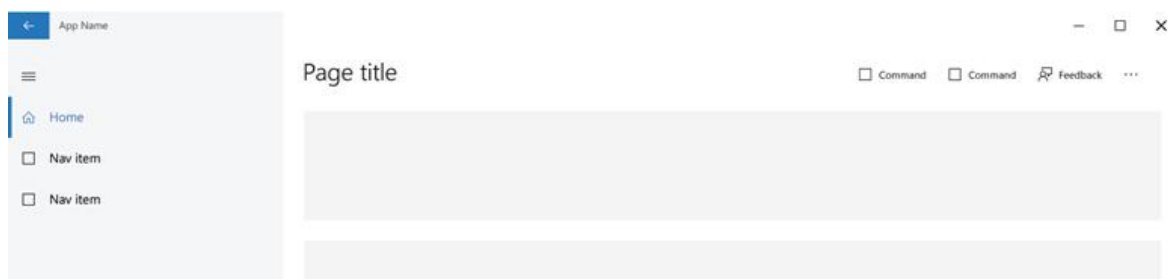
Design recommendations for your feedback UI

To launch Feedback Hub, we recommend that you add a UI element in your app (such as a button) that displays the following standard feedback icon from the Segoe MDL2 Assets font and the character code E939.



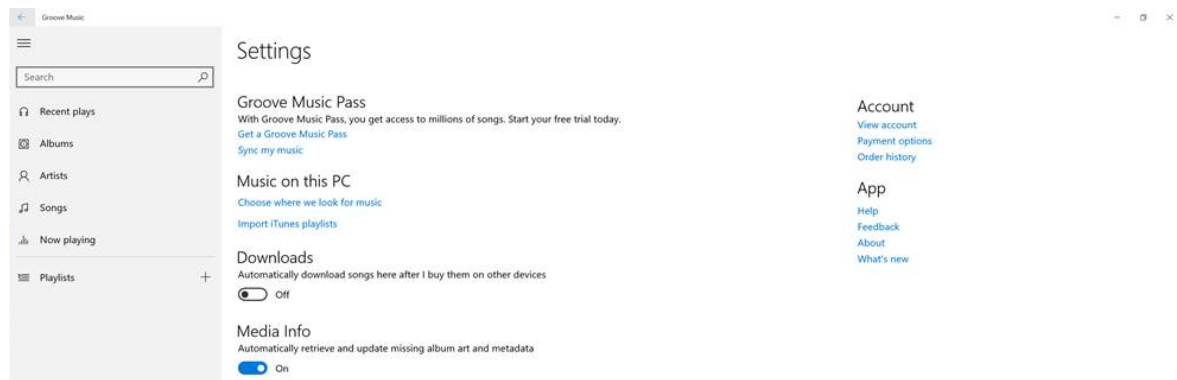
We also recommend that you use one or more of the following placement options for linking to Feedback Hub in your app.

- **Directly in the app bar.** Depending on your implementation, you may wish to use the icon only or add text (as shown below).



- **In your app's settings.** This is a more subtle way to provide access to Feedback Hub. In the example below, the Feedback link appears as one of the links under

App.



- **In an event-driven flyout.** This is useful when you want to query your customers about a specific question before launching into the Windows Feedback Hub. For example, after your app uses a certain feature, you might prompt the customer with a specific question about their satisfaction with that feature. If the customer chooses to respond, your app launches Feedback Hub.

Related topics

- [Feedback report](#)

Configure your app for targeted push notifications

Article • 01/04/2023

You can use the **Push notifications** page in Partner Center to directly engage with customers by sending targeted push notifications to the devices on which your Universal Windows Platform (UWP) app is installed. For example, you can use targeted push notifications to encourage your customers to take an action, such as rating your app or trying a new feature. You can send several different types of push notifications, including toast notifications, tile notifications, and raw XML notifications. You can also track the rate of app launches that resulted from your push notifications. For more information about this feature, see [Send push notifications to your app's customers](#).

Before you can send targeted push notifications to your customers from Partner Center, you must use a method of the [StoreServicesEngagementManager](#) class in the Microsoft Store Services SDK to register your app to receive notifications. You can use additional methods of this class to notify Partner Center that your app was launched in response to a targeted push notification (if you want to track the rate of app launches that resulted from your notifications) and to stop receiving notifications.

Configure your project

Before you write any code, follow these steps to add a reference to the Microsoft Store Services SDK in your project:

1. If you have not done so already, [Install the Microsoft Store Services SDK](#) on your development computer.
2. Open your project in Visual Studio.
3. In Solution Explorer, right-click the **References** node for your project and click **Add Reference**.
4. In **Reference Manager**, expand **Universal Windows** and click **Extensions**.
5. In the list of SDKs, click the check box next to **Microsoft Engagement Framework** and click **OK**.

Register for push notifications

To register your app to receive targeted push notifications from Partner Center:

1. In your project, locate a section of code that runs during startup in which you can register your app to receive notifications.
2. Add the following statement to the top of the code file.

C#

```
using Microsoft.Services.Store.Engagement;
```

3. Get a [StoreServicesEngagementManager](#) object and call one of the [RegisterNotificationChannelAsync](#) overloads in the startup code you identified earlier. This method should be called each time that your app is launched.
- If you want Partner Center to create its own channel URI for the notifications, call the [RegisterNotificationChannelAsync\(\)](#) overload.

C#

```
StoreServicesEngagementManager engagementManager =  
StoreServicesEngagementManager.Default();  
await engagementManager.RegisterNotificationChannelAsync();
```

Important

If your app also calls [CreatePushNotificationChannelForApplicationAsync](#) to create a notification channel for WNS, make sure that your code does not call [CreatePushNotificationChannelForApplicationAsync](#) and the [RegisterNotificationChannelAsync\(\)](#) overload simultaneously. If you need to call both of these methods, make sure that you call them sequentially and await the return of one method before calling the other.

- If you want to specify the channel URI to use for targeted push notifications from Partner Center, call the [RegisterNotificationChannelAsync\(StoreServicesNotificationChannelParameters\)](#) overload. For example, might want to do this if your app already uses Windows Push Notification Services (WNS) and you want to use the same channel URI. You must first create a [StoreServicesNotificationChannelParameters](#) object and assign the [CustomNotificationChannelUri](#) property to your channel URI.

C#

```
StoreServicesNotificationChannelParameters parameters =  
new StoreServicesNotificationChannelParameters();
```

```
parameters.CustomNotificationChannelUri = "Assign your channel URI here";

StoreServicesEngagementManager engagementManager =
StoreServicesEngagementManager.Default();
await engagementManager.RegisterNotificationChannelAsync(parameters);
```

ⓘ Note

When you call the **RegisterNotificationChannelAsync** method, a file named **MicrosoftStoreEngagementSDKId.txt** is created in the local app data store for your app (the folder returned by the **ApplicationData.LocalFolder** property). This file contains an ID that is used by the targeted push notifications infrastructure. Make sure that your app does not modify or delete this file. Otherwise, your users may receive multiple instances of notifications, or the notifications may not behave properly in other ways.

How targeted push notifications are routed to customers

When your app calls **RegisterNotificationChannelAsync**, this method collects the Microsoft account of the customer who is currently signed in to the device. Later, when you send a targeted push notification to a segment that includes this customer, Partner Center sends the notification to devices that are associated with this customer's Microsoft account.

If the customer who started your app gives their device to someone else to use while they are still signed in to the device with their Microsoft account, be aware that the other person may see the notification that was targeted at the original customer. This can have unintended consequences, particularly for apps that offer services that customers can sign in to use. To prevent other users from seeing your targeted notifications in this scenario, call the **UnregisterNotificationChannelAsync** method when customers sign out of your app. For more information, see [Unregister for push notifications](#) later in this article.

How your app responds when the user launches your app

After your app is registered to receive notifications and you [send a push notification to your app's customers from Partner Center](#), one of the following entry points in your app will be called when the user launches your app in response to your push notification. If you have some code that you want to run when the user launches your app, you can add the code to one of these entry points in your app.

- If the push notification has a foreground activation type, override the [OnActivated](#) method of the **App** class in your project and add your code to this method.
- If the push notification has a background activation type, add your code to the [Run](#) method for your [background task](#).

For example, you might want to reward the users of your app that have purchased any paid add-ons in your app by granting them a free add-on. In this case, you can send a push notification to a [customer segment](#) that targets these users. Then, you can add code to grant them a free [in-app purchase](#) in one of the entry points listed above.

Notify Partner Center of your app launch

If you select the **Track app launch rate** option for your targeted push notification in Partner Center, call the [ParseArgumentsAndTrackAppLaunch](#) method from the appropriate entry point in your app to notify Partner Center that your app was launched in response to a push notification.

This method also returns the original launch arguments for your app. When you choose to track the app launch rate for your push notification, an opaque tracking ID is added to the launch arguments to help track the app launch in Partner Center. You must pass the launch arguments for your app to the [ParseArgumentsAndTrackAppLaunch](#) method, and this method sends the tracking ID to Partner Center, removes the tracking ID from the launch arguments, and returns the original launch arguments to your code.

The way you call this method depends on the activation type of the push notification:

- If the push notification has a foreground activation type, call this method from the [OnActivated](#) method override in your app and pass the arguments that are available in the [ToastNotificationActivatedEventArgs](#) object that is passed to this method. The following code example assumes that your code file has **using** statements for the **Microsoft.Services.Store.Engagement** and **Windows.ApplicationModel.Activation** namespaces.

C#

```
protected override void OnActivated(IActivatedEventArgs args)
{
    base.OnActivated(args);

    if (args is ToastNotificationActivatedEventArgs)
    {
        var toastActivationArgs = args as
        ToastNotificationActivatedEventArgs;
```

```

        StoreServicesEngagementManager engagementManager =
        StoreServicesEngagementManager.Default();
        string originalArgs =
        engagementManager.ParseArgumentsAndTrackAppLaunch(
            toastActivationArgs.Argument);

        // Use the originalArgs variable to access the original
        arguments
        // that were passed to the app.
    }
}

```

- If the push notification has a background activation type, call this method from the [Run](#) method for your [background task](#) and pass the arguments that are available in the [ToastNotificationActionTriggerDetail](#) object that is passed to this method. The following code example assumes that your code file has [using](#) statements for the [Microsoft.Services.Store.Engagement](#), [Windows.ApplicationModel.Background](#), and [Windows.UI.Notifications](#) namespaces.

```

C#

public void Run(IBackgroundTaskInstance taskInstance)
{
    var details = taskInstance.TriggerDetails as
    ToastNotificationActionTriggerDetail;

    if (details != null)
    {
        StoreServicesEngagementManager engagementManager =
        StoreServicesEngagementManager.Default();
        string originalArgs =
        engagementManager.ParseArgumentsAndTrackAppLaunch(details.Argument);

        // Use the originalArgs variable to access the original
        arguments
        // that were passed to the app.
    }
}

```

Unregister for push notifications

If you want your app to stop receiving targeted push notifications from Partner Center, call the [UnregisterNotificationChannelAsync](#) method.

C#

```

StoreServicesEngagementManager engagementManager =
StoreServicesEngagementManager.Default();

```



```
await engagementManager.UnregisterNotificationChannelAsync();
```

Note that this method invalidates the channel that is being used for notifications so the app no longer receives push notifications from *any* services. After it has been closed, the channel cannot be used again for any services, including targeted push notifications from Partner Center and other notifications using WNS. To resume sending push notifications to this app, the app must request a new channel.

Related topics

- [Send push notifications to your app's customers](#)
- [Windows Push Notification Services \(WNS\) overview](#)
- [How to request, create, and save a notification channel](#)
- [Microsoft Store Services SDK](#)

Log custom events for Partner Center

Article • 11/01/2022

The [Usage report](#) in Partner Center lets you get info about custom events that you've defined in your Universal Windows Platform (UWP) app. A custom event is an arbitrary string that represents an event or activity in your app. For example, a game might define custom events named *firstLevelPassed*, *secondLevelPassed*, and so on, which are logged when the user passes each level in the game.

To log a custom event from your app, pass the custom event string to the [Log](#) method provided by the Microsoft Store Services SDK. You can review the total occurrences for your custom events in the **Custom events** section of the [Usage report](#) in Partner Center.

ⓘ Note

Custom events that you log to Partner Center are unrelated to **Windows events**, and they do not appear in **Event Viewer**.

Prerequisites

Before you can review custom logging events in the **Usage report** for your app in Partner Center, your app must be published in the Store.

How to log custom events

1. If you have not done so already, [Install the Microsoft Store Services SDK](#) on your development computer.
2. Open your project in Visual Studio.
3. In Solution Explorer, right-click the **References** node for your project and click **Add Reference**.
4. In **Reference Manager**, expand **Universal Windows** and click **Extensions**.
5. In the list of SDKs, click the check box next to **Microsoft Engagement Framework** and click **OK**.
6. Add the following statement to the top of each code file where you want to log custom events.

C#

```
using Microsoft.Services.Store.Engagement;
```

7. In each section of your code where you want to log a custom event, get a [StoreServicesCustomEventLogger](#) object and then call the [Log](#) method. Pass your custom event string to the method.

C#

```
StoreServicesCustomEventLogger logger =  
StoreServicesCustomEventLogger.Default;  
logger.Log("myCustomEvent");
```

ⓘ Note

The **Usage report** may take a long time to load if your app logs many custom events with long names. We recommend that you use brief names for your custom events.

Related topics

- [Usage report](#)
- [Log method](#)
- [Microsoft Store Services SDK](#)

Request ratings and reviews for your app

Article • 11/01/2022

You can add code to your Universal Windows Platform (UWP) app to programmatically prompt your customers to rate or review your app. There are several ways you can do this:

- You can show a rating and review dialog directly in the context of your app.
- You can programmatically open the rating and review page for your app in the Microsoft Store.

When you are ready to analyze your ratings and reviews data, you can view the data in Partner Center or use the Microsoft Store analytics API to retrieve this data programmatically.

Important

When adding a rating function within your app, all reviews must send the user to the Store's rating mechanisms, regardless of star rating chosen. If you collect feedback or comments from users, it must be clear that it is not related to the app rating or reviews in the Store but is sent directly to the app developer. See the Developer Code of Conduct for more information related to [Fraudulent or Dishonest Activities](#).

Show a rating and review dialog in your app

To programmatically show a dialog from your app that asks your customer to rate your app and submit a review, call the [RequestRateAndReviewAppAsync](#) method in the [Windows.Services.Store](#) namespace.

Important

The request to show the rating and review dialog must be called on the UI thread in your app.

C#

```

using Windows.ApplicationModel.Store;

private StoreContext _storeContext;

public async Task Initialize()
{
    if (App.IsMultiUserApp) // pseudo-code
    {
        IReadOnlyList<User> users = await User.FindAllAsync();
        User firstUser = users[0];
        _storeContext = StoreContext.GetForUser(firstUser);
    }
    else
    {
        _storeContext = StoreContext.Default();
    }
}

private async Task PromptUserToRateApp()
{
    // Check if we've recently prompted user to review, we don't want to
    // bother user too often and only between version changes
    if (HaveWePromptedUserInPastThreeMonths()) // pseudo-code
    {
        return;
    }

    StoreRateAndReviewResult result = await
        _storeContext.RequestRateAndReviewAppAsync();

    // Check status
    switch (result.Status)
    {
        case StoreRateAndReviewStatus.Succeeded:
            // Was this an updated review or a new review, if Updated is
            // false it means it was a users first time reviewing
            if (result.UpdatedExistingRatingOrReview)
            {
                // This was an updated review thank user
                ThankUserForReview(); // pseudo-code
            }
            else
            {
                // This was a new review, thank user for reviewing and give
                // some free in app tokens
                ThankUserForReviewAndGrantTokens(); // pseudo-code
            }
            // Keep track that we prompted user and don't do it again for a
            // while
            SetUserHasBeenPrompted(); // pseudo-code
            break;

        case StoreRateAndReviewStatus.CanceledByUser:
            // Keep track that we prompted user and don't prompt again for a

```

```

while
    SetUserHasBeenPrompted(); // pseudo-code

    break;

    case StoreRateAndReviewStatus.NetworkError:
        // User is probably not connected, so we'll try again, but keep
track so we don't try too often
        SetUserHasBeenPromptedButHadNetworkError(); // pseudo-code

        break;

        // Something else went wrong
    case StoreRateAndReviewStatus.OtherError:
    default:
        // Log error, passing in ExtendedJsonData however it will be
empty for now
        LogError(result.ExtendedError, result.ExtendedJsonData); //
pseudo-code
        break;
    }
}

```

The **RequestRateAndReviewAppAsync** method was introduced in Windows 10, version 1809, and it can only be used in projects that target **Windows 10 October 2018 Update (10.0; Build 17763)** or a later release in Visual Studio.

Response data for the rating and review request

After you submit the request to display the rating and review dialog, the [ExtendedJsonData](#) property of the [StoreRateAndReviewResult](#) class contains a JSON-formatted string that indicates whether the request was successful.

The following example demonstrates the return value for this request after the customer successfully submits a rating or review.

JSON

```

{
  "status": "success",
  "data": {
    "updated": false
  },
  "errorDetails": "Success"
}

```

The following example demonstrates the return value for this request after the customer chooses not to submit a rating or review.

JSON

```
{
  "status": "aborted",
  "errorDetails": "Navigation was unsuccessful"
}
```

The following table describes the fields in the JSON-formatted data string.

Field	Description
<i>status</i>	A string that indicates whether the customer successfully submitted a rating or review. The supported values are success and aborted .
<i>data</i>	An object that contains a single Boolean value named <i>updated</i> . This value indicates whether the customer updated an existing rating or review. The <i>data</i> object is included in success responses only.
<i>errorDetails</i>	A string that contains the error details for the request.

Launch the rating and review page for your app in the Store

If you want to programmatically open the rating and review page for your app in the Store, you can use the [LaunchUriAsync](#) method with the `ms-windows-store://review` URI scheme as demonstrated in this code example.

C#

```
bool result = await Windows.System.Launcher.LaunchUriAsync(new Uri("ms-windows-store://review/?ProductId=9WZDNCRFHVJL"));
```

For more information, see [Launch the Microsoft Store app](#).

Analyze your ratings and reviews data

To analyze the ratings and reviews data from your customers, you have several options:

- You can use the [Reviews](#) report in Partner Center to see the ratings and reviews from your customers. You can also download this report to view it offline.
- You can use the [Get app ratings](#) and [Get app reviews](#) methods in the Store analytics API to programmatically retrieve the ratings and reviews from your customers in JSON format.

Related topics

- [Send requests to the Store](#)
- [Launch the Microsoft Store app](#)
- [Reviews report](#)

Microsoft Store services

Article • 10/20/2022

The Microsoft Store exposes several services that you can call via REST APIs to programmatically access certain types of data for apps that are registered to your or your organization's Partner Center account.

In this section

Topic	Description
Access analytics data	Use the <i>Microsoft Store analytics API</i> to programmatically retrieve analytics data for your apps. This API enables you to retrieve data for app and add-on (also known as in-app product or IAP) acquisitions, app failures, app ratings and reviews, and performance data for in-app ads and promotional ad campaigns.
Respond to reviews	Use the <i>Microsoft Store reviews API</i> to programmatically respond to reviews of your app in the Store. This API is especially useful for developers who want to bulk respond to many reviews without using Partner Center.
Run ad campaigns	Use the <i>Microsoft Store promotions API</i> to programmatically manage promotional ad campaigns for your apps. This API enables you to create, update and monitor your campaigns and other related assets such as targeting and creatives. This API is especially useful for developers who create large volumes of campaigns, and who want to do so without using Partner Center dashboard.
Create and manage submissions	Use the <i>Microsoft Store submission API</i> to programmatically query and create submissions for apps, add-ons and package flights for your or your organization's Partner Center account. This API is useful if your account manages many apps or add-ons, and you want to automate and optimize the submission process for these assets.
Manage targeted offers	Use the <i>Microsoft Store targeted offers API</i> to programmatically claim a targeted offer that is associated with the successful purchase of an add-on in your app.
Manage product entitlements from a service	If you have a catalog of apps and add-ons in the Store, you can use the <i>Microsoft Store collection API</i> and <i>Microsoft Store purchase API</i> to access ownership information for these products from your services, report a consumable product as fulfilled for a user, and grant an entitlement for a free product to a user.

Access analytics data using Store services

Article • 06/13/2024

Use the *Microsoft Store analytics API* to programmatically retrieve analytics data for apps that are registered to your or your organization's Windows Partner Center account. This API enables you to retrieve data for app and add-on (also known as in-app product or IAP) acquisitions, errors, app ratings and reviews. This API uses Microsoft Entra to authenticate the calls from your app or service.

The following steps describe the end-to-end process:

1. Make sure that you have completed all the [prerequisites](#).
2. Before you call a method in the Microsoft Store analytics API, [obtain a Microsoft Entra access token](#). After you obtain a token, you have 60 minutes to use this token in calls to the Microsoft Store analytics API before the token expires. After the token expires, you can generate a new token.
3. [Call the Microsoft Store analytics API](#).

Step 1: Complete prerequisites for using the Microsoft Store analytics API

Before you start writing code to call the Microsoft Store analytics API, make sure that you have completed the following prerequisites.

- You (or your organization) must have a Microsoft Entra directory and you must have [Global administrator](#) permission for the directory. If you already use Microsoft 365 or other business services from Microsoft, you already have a Microsoft Entra directory. Otherwise, you can [create a new one in Partner Center](#) for no additional charge.
- You must associate a Microsoft Entra application with your Partner Center account, retrieve the tenant ID and client ID for the application and generate a key. The Microsoft Entra application represents the app or service from which you want to call the Microsoft Store analytics API. You need the tenant ID, client ID and key to obtain a Microsoft Entra access token that you pass to the API.

ⓘ Note

You only need to perform this task one time. After you have the tenant ID, client ID and key, you can reuse them any time you need to create a new Microsoft Entra access token.

To associate a Microsoft Entra application with your Partner Center account and retrieve the required values:

1. In Partner Center, [associate your organization's Partner Center account with your organization's Microsoft Entra directory](#).
2. Next, from the **Users** page in the **Account settings** section of Partner Center, [add the Microsoft Entra application](#) that represents the app or service that you will use to access analytics data for your Partner Center account. Make sure you assign this application the **Manager** role. If the application doesn't exist yet in your Microsoft Entra directory, you can [create a new Microsoft Entra application in Partner Center](#).
3. Return to the **Users Management** page and navigate to **Microsoft Entra applications** tab, click the name of your Microsoft Entra application to go to the application settings, and copy down the **Tenant ID** and **Client ID** values.
4. Click **Add new key**. On the following screen, copy down the **Key** value. You won't be able to access this info again after you leave this page. For more information, see [Manage keys for a Microsoft Entra application](#).

Step 2: Obtain a Microsoft Entra access token

Before you call any of the methods in the Microsoft Store analytics API, you must first obtain a Microsoft Entra access token that you pass to the **Authorization** header of each method in the API. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can refresh the token so you can continue to use it in further calls to the API.

To obtain the access token, follow the instructions in [Service to Service Calls Using Client Credentials](#) to send an HTTP POST to the

`https://login.microsoftonline.com/<tenant_id>/oauth2/token` endpoint. Here is a sample request.

JSON

```
POST https://login.microsoftonline.com/<tenant_id>/oauth2/token HTTP/1.1
Host: login.microsoftonline.com
Content-Type: application/x-www-form-urlencoded; charset=utf-8

grant_type=client_credentials
&client_id=<your_client_id>
```

```
&client_secret=<your_client_secret>
&resource=https://manage.devcenter.microsoft.com
```

For the *tenant_id* value in the POST URI and the *client_id* and *client_secret* parameters, specify the tenant ID, client ID and the key for your application that you retrieved from Partner Center in the previous section. For the *resource* parameter, you must specify `https://manage.devcenter.microsoft.com`.

After your access token expires, you can refresh it by following the instructions [here](#).

ⓘ Note

ResourceType= `Graph.windows.net` was deprecated in September 2023. Please migrate to ResourceType = `Graph.microsoft.com`.

Step 3: Call the Microsoft Store analytics API

After you have a Microsoft Entra access token, you are ready to call the Microsoft Store analytics API. You must pass the access token to the **Authorization** header of each method.

Methods for UWP apps and games

The following methods are available for apps and games acquisitions and add-on acquisitions:

- [Get acquisitions data for your games and apps](#)
- [Get add-on acquisitions data for your games and apps](#)

Methods for UWP apps

The following analytics methods are available for UWP apps in Partner Center.

[Expand table](#)

Scenario	Methods
Acquisitions, conversions, installs, and usage	• Get app acquisitions (legacy) • Get app acquisition funnel data (legacy) • Get app conversions by channel • Get add-on acquisitions • Get subscription add-on acquisitions

Scenario	Methods
	• Get add-on conversions by channel • Get app installs • Get daily app usage • Get monthly app usage
App errors	• Get error reporting data • Get details for an error in your app • Get the stack trace for an error in your app • Download the CAB file for an error in your app
Insights	• Get insights data for your app
Ratings and reviews	• Get app ratings • Get app reviews
In-app ads and ad campaigns	• Get ad performance data • Get ad campaign performance data

Methods for desktop applications

The following analytics methods are available for use by developer accounts that belong to the [Windows Desktop Application program](#).

 Expand table

Scenario	Methods
Installs	• Get desktop application installs
Blocks	• Get upgrade blocks for your desktop application • Get upgrade block details for your desktop application
Application errors	• Get error reporting data for your desktop application • Get details for an error in your desktop application • Get the stack trace for an error in your desktop application • Download the CAB file for an error in your desktop application
Insights	• Get insights data for your desktop application

Methods for Xbox Live services

The following additional methods are available for use by developer accounts with games that use [Xbox Live services](#). The Microsoft Store Analytics API for Xbox is no longer available. [gaming/xbox-live/get-started/join-dev-program/join-dev-program_nav](#)

 Expand table

Scenario	Methods
General analytics	• Get Xbox Live analytics data

Methods for hardware and drivers

Developer accounts that belong to the [Windows hardware dashboard program](#) have access to an additional set of methods for retrieving analytics data for hardware and drivers. For more information, see [Hardware dashboard API](#).

Code example

The following code example demonstrates how to obtain a Microsoft Entra access token and call the Microsoft Store analytics API from a C# console app. To use this code example, assign the *tenantId*, *clientId*, *clientSecret*, and *appId* variables to the appropriate values for your scenario. This example requires the [Json.NET package](#)  from Newtonsoft to deserialize the JSON data returned by the Microsoft Store analytics API.

C#

```
using Newtonsoft.Json;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Text;
using System.Threading.Tasks;

namespace TestAnalyticsAPI
{
    class Program
    {
        static void Main(string[] args)
        {
            string tenantId = "<your tenant ID>";
```

```

string clientId = "<your client ID>";
string clientSecret = "<your secret>";

string scope = "https://manage.devcenter.microsoft.com";

// Retrieve an Azure AD access token
string accessToken = GetClientCredentialAccessToken(
    tenantId,
    clientId,
    clientSecret,
    scope).Result;

// This is your app's Store ID. This ID is available on
// the App identity page of the Dev Center dashboard.
string appID = "<your app's Store ID>";

DateTime startDate = DateTime.Parse("08-01-2015");
DateTime endDate = DateTime.Parse("11-01-2015");
int pageSize = 1000;
int startPageIndex = 0;

// Call the Windows Store analytics API
CallAnalyticsAPI(accessToken, appID, startDate, endDate,
pageSize, startPageIndex);

Console.Read();
}

private static void CallAnalyticsAPI(string accessToken, string
appID, DateTime startDate, DateTime endDate, int top, int skip)
{
    string requestURI;

    // Get app acquisitions
    requestURI = string.Format(

"https://manage.devcenter.microsoft.com/v1.0/my/analytics/appacquisitions?
applicationId={0}&startDate={1}&endDate={2}&top={3}&skip={4}",
        appID, startDate, endDate, top, skip);

    //// Get add-on acquisitions
    //requestURI = string.Format(
    //
"https://manage.devcenter.microsoft.com/v1.0/my/analytics/inappacquisitions?
applicationId={0}&startDate={1}&endDate={2}&top={3}&skip={4}",
        appID, startDate, endDate, top, skip);

    //// Get app failures
    //requestURI = string.Format(
    //
"https://manage.devcenter.microsoft.com/v1.0/my/analytics/failurehits?
applicationId={0}&startDate={1}&endDate={2}&top={3}&skip={4}",
        appID, startDate, endDate, top, skip);

    //// Get app ratings

```

```

        //requestURI = string.Format(
        //
        "https://manage.devcenter.microsoft.com/v1.0/my/analytics/ratings?
applicationId={0}&startDate={1}&endDate={2}&top={3}&skip={4}",
        //    appID, startDate, endDate, top, skip);

        //// Get app reviews
        //requestURI = string.Format(
        //
        "https://manage.devcenter.microsoft.com/v1.0/my/analytics/reviews?
applicationId={0}&startDate={1}&endDate={2}&top={3}&skip={4}",
        //    appID, startDate, endDate, top, skip);

        HttpRequestMessage requestMessage = new
HttpRequestMessage(HttpMethod.Get, requestURI);
        requestMessage.Headers.Authorization = new
AuthenticationHeaderValue("Bearer", accessToken);

        WebRequestHandler handler = new WebRequestHandler();
        HttpClient httpClient = new HttpClient(handler);

        HttpResponseMessage response =
httpClient.SendAsync(requestMessage).Result;

        Console.WriteLine(response);
        Console.WriteLine(response.Content.ReadAsStringAsync().Result);

        response.Dispose();
    }

    public static async Task<string>
GetClientCredentialAccessToken(string tenantId, string clientId, string
clientSecret, string scope)
    {
        string tokenEndpointFormat =
"https://login.microsoftonline.com/{0}/oauth2/token";
        string tokenEndpoint = string.Format(tokenEndpointFormat,
tenantId);

        dynamic result;
        using (HttpClient client = new HttpClient())
        {
            string tokenUrl = tokenEndpoint;
            using (
                HttpRequestMessage request = new HttpRequestMessage(
                    HttpMethod.Post,
                    tokenUrl))
            {
                string content =
                    string.Format(
                        "grant_type=client_credentials&client_id=
{0}&client_secret={1}&resource={2}",
                        clientId,
                        clientSecret,
                        scope);
            }
        }
    }

```



```

        request.Content = new StringContent(content,
Encoding.UTF8, "application/x-www-form-urlencoded");

        using (HttpResponseMessage response = await
client.SendAsync(request))
        {
            string responseContent = await
response.Content.ReadAsStringAsync();
            result =
JsonConvert.DeserializeObject(responseContent);
        }
    }

    return result.access_token;
}
}
}

```

Error responses

The Microsoft Store analytics API returns error responses in a JSON object that contains error codes and messages. The following example demonstrates an error response caused by an invalid parameter.

JSON

```

{
  "code": "BadRequest",
  "data": [],
  "details": [],
  "innererror": {
    "code": "InvalidQueryParameters",
    "data": [
      "top parameter cannot be more than 10000"
    ],
    "details": [],
    "message": "One or More Query Parameters has invalid values.",
    "source": "AnalyticsAPI"
  },
  "message": "The calling client sent a bad request to the service.",
  "source": "AnalyticsAPI"
}

```

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

| [Get help at Microsoft Q&A](#)

Get acquisitions data for your games and apps

Article • 02/06/2023

Use this method in the Microsoft Store analytics API to get aggregate acquisition data in JSON format for UWP apps and Xbox One games that were ingested through the Xbox Developer Portal (XDP) and available in the XDP Analytics dashboard.

ⓘ Note

This API does not provide daily aggregate data before Oct 1st 2016.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store analytics API.
- [Obtain an Azure AD access token](#) to use in the request header for this method.
After you obtain an access token, you have 60 minutes to use it before it expires.
After the token expires, you can obtain a new one.

Request

Request syntax

Method	Request URI
GET	<code>https://manage.devcenter.microsoft.com/v1.0/my/analytics/acquisitions</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <code><token></code> .

Request parameters

Parameter	Type	Description	Required
applicationId	string	The product ID of the Xbox One game for which you are retrieving acquisition data. To get the product ID of your game, navigate to your game in the XDP Analytics Program and retrieve the product ID from the URL. Alternatively, if you download your acquisitions data from the Partner Center analytics report, the product ID is included in the .tsv file.	Yes
startDate	date	The start date in the date range of acquisition data to retrieve. The default is the current date.	No
endDate	date	The end date in the date range of acquisition data to retrieve. The default is the current date.	No
filter	string	One or more statements that filter the rows in the response. Each statement contains a field name from the response body and value that are associated with the eq or ne operators, and statements can be combined using and or or. String values must be surrounded by single quotes in the filter parameter. For example, <i>filter=market eq 'US' and gender eq 'm'</i>. You can specify the following fields from the response body: • acquisitionType • age • storeClient • gender • market • osVersion • deviceType • sandboxId	No
aggregationLevel	string	Specifies the time range for which to retrieve aggregate data. Can be one of the following strings: day , week , or month . If unspecified, the default is day .	No

Parameter	Type	Description	Required
orderby	string	A statement that orders the result data values for each acquisition. The syntax is <i>orderby=field [order],field [order],...</i> The <i>field</i> parameter can be one of the following strings: • date • acquisitionType • age • storeClient • gender • market • osVersion • deviceType • paymentInstrumentType • sandboxId • xboxTitleId The <i>order</i> parameter is optional, and can be asc or desc to specify ascending or descending order for each field. The default is asc. Here is an example <i>orderby</i> string: <i>orderby=date,market</i>	No
groupby	string	A statement that applies data aggregation only to the specified fields. You can specify the following fields: • date • applicationName • acquisitionType • age • storeClient • gender • market • osVersion • deviceType • paymentInstrumentType • sandboxId • xboxTitleId The returned data rows will contain the fields specified in the <i>groupby</i> parameter as well as the following: • date • applicationId • acquisitionQuantity The <i>groupby</i> parameter can be used with the <i>aggregationLevel</i> parameter. For example: <i>&groupby=age,market&aggregationLevel=week</i>	No

Request example

The following example demonstrates several requests for getting Xbox One game acquisition data. Replace the *applicationId* value with the product ID for your game.

syntax

```
GET https://manage.devcenter.microsoft.com/v1.0/my/analytics/acquisitions?
applicationId=9WZDNCRFHXHT&startDate=1/1/2017&endDate=2/1/2019&top=10&skip=0
HTTP/1.1
```

```
Authorization: Bearer <your access token>
```

```
GET https://manage.devcenter.microsoft.com/v1.0/my/analytics/acquisitions?
applicationId=9WZDNCRFHXHT&startDate=1/1/2017&endDate=2/1/2019&skip=0&filter
=market eq 'US' and gender eq 'm' HTTP/1.1
```

```
Authorization: Bearer <your access token>
```

Response

Response body

Value	Type	Description
Value	array	An array of objects that contain aggregate acquisition data for the game. For more information about the data in each object, see the acquisition values section below.
TotalCount	integer	The total number of rows in the data result for the query.

Acquisition values

Elements in the *Value* array contain the following values.

Value	Type	Description
date	string	The first date in the date range for the acquisition data. If the request specified a single day, this value is that date. If the request specified a week, month, or other date range, this value is the first date in that date range.
applicationId	string	The product ID of the Xbox One game for which you are retrieving acquisition data.
applicationName	string	The display name of the game.

Value	Type	Description
acquisitionType	string	One of the following strings that indicates the type of acquisition: • Free • Trial • Paid • Promotional code • lap • Subscription lap • Private Audience • Pre Order • Xbox Game Pass (or Game Pass if querying for data before March 23, 2018) • Disk • Prepaid Code • Charged Pre Order • Cancelled Pre Order • Failed Pre Order
age	string	One of the following strings that indicates the age group of the user who made the acquisition: • Less than 13 • 13-17 • 18-24 • 25-34 • 35-44 • 44-55 • Greater than 55 • Unknown
deviceType	string	One of the following strings that specifies the type of device that completed the acquisition: • PC • Phone • Console-Xbox One • Console-Xbox Series X • IoT • Server • Tablet • Holographic • Unknown

Value	Type	Description
gender	string	One of the following strings that specifies the gender of the user who made the acquisition: • m • f • Unknown
market	string	The ISO 3166 country code of the market where the acquisition occurred.
osVersion	string	The OS version on which the acquisition occurred. For this method, this value is always either Windows 10 or Windows 11 .
paymentInstrumentType	string	One of the following strings that indicates the payment instruction used for the acquisition: • Credit Card • Direct Debit Card • Inferred Purchase • MS Balance • Mobile Operator • Online Bank Transfer • PayPal • Split Transaction • Token Redemption • Zero Amount Paid • eWallet • Unknown
sandboxId	string	The sandbox ID created for the game. This can be the value RETAIL or a private sandbox ID.
storeClient	string	One of the following strings that indicates the version of the Store where the acquisition occurred: • Windows Phone Store (client) • Microsoft Store (client) (or Windows Store (client) if querying for data before March 23, 2018) • Microsoft Store (web) (or Windows Store (web) if querying for data before March 23, 2018) • Volume purchase by organizations • Other
xboxTitleId	string	The Xbox Live title ID (represented in hexadecimal value) assigned by the Xbox Developer Portal (XDP) for Xbox Live-enabled games.

Value	Type	Description
acquisitionQuantity	number	The number of acquisitions that occurred during the specified aggregation level.
purchasePriceUSDAmount	number	The amount paid by the customer for the acquisition, converted to USD, using the monthly exchange rate.
purchaseTaxUSDAmount	number	The tax amount applied to the acquisition, converted to USD.
localCurrencyCode	string	Local Currency code based on the country of the Partner Center account.
xboxProductId	string	Xbox Product ID of the product from XDP, if applicable.
availabilityId	string	Availability ID of the product from XDP, if applicable.
skuld	string	SKU ID of the product from XDP, if applicable.
skuDisplayName	string	SKU display name of the product from XDP, if applicable.
xboxParentProductId	string	Xbox Parent Product ID of the product from XDP, if applicable.
parentProductName	string	Parent Product Name of the product from XDP, if applicable.
productTypeName	string	Product Type Name of the product from XDP, if applicable.
purchaseTaxType	string	Purchase Tax Type of the product from XDP, if applicable.
purchasePriceLocalAmount	number	Purchase Price Local Amount of the product from XDP, if applicable.
purchaseTaxLocalAmount	number	Purchase Tax Local Amount of the product from XDP, if applicable.

Response example

The following example demonstrates an example JSON response body for this request.

JSON

```
{
  "Value": [
    {
      "date": "2019-01-15T01:00:00.000000Z",
      "applicationId": "9WZDNCRFHXT",
      "applicationName": null,

```

```
    "acquisitionType": "Paid",
    "age": null,
    "deviceType": "Phone",
    "gender": null,
    "market": "US",
    "osVersion": "Windows 11",
    "paymentInstrumentType": null,
    "sandboxId": "RETAIL",
    "storeClient": "Microsoft Store (client)",
    "xboxTitleId": null,
    "localCurrencyCode": "USD",
    "xboxProductId": null,
    "availabilityId": "B42LRTSZ2MCJ",
    "skuId": "0010",
    "skuDisplayName": null,
    "xboxParentProductId": null,
    "parentProductName": null,
    "productTypeName": "Game",
    "purchaseTaxType": "TaxesNotIncluded",
    "acquisitionQuantity": 1,
    "purchasePriceUSDAmount": 3.08,
    "purchasePriceLocalAmount": 3.08,
    "purchaseTaxUSDAmount": 0.09,
    "purchaseTaxLocalAmount": 0.09
  },
],

"@nextLink": null,

"TotalCount": 12221
}
```

Get add-on acquisitions data for your games and apps

Article • 10/20/2022

Use this method in the Microsoft Store analytics API to get aggregate add-on acquisition data in JSON format for UWP apps and Xbox One games that were ingested through the Xbox Developer Portal (XDP) and available in the XDP Analytics Partner Center dashboard.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store analytics API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.

Note

This API does not provide daily aggregate data before Oct 1st 2016.

Request

Request syntax

Method	Request URI
GET	<code>https://manage.devcenter.microsoft.com/v1.0/my/analytics/addonacquisitions</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <code><token></code> .

Request parameters

The *applicationId* or *addonProductId* parameter is required. To retrieve acquisition data for all add-ons registered to the app, specify the *applicationId* parameter. To retrieve acquisition data for a single add-on, specify the *addonProductId* parameter. If you specify both, the *applicationId* parameter is ignored.

Parameter	Type	Description	Required
applicationId	string	The <i>productId</i> of the Xbox One game for which you are retrieving acquisition data. To get the <i>productId</i> of your game, navigate to your game in the XDP Analytics Program and retrieve the <i>productId</i> from the URL. Alternatively, if you download your acquisitions data from the Partner Center analytics report, the <i>productId</i> is included in the .tsv file.	Yes
addonProductId	string	The <i>productId</i> of the add-on for which you want to retrieve acquisition data.	Yes
startDate	date	The start date in the date range of add-on acquisition data to retrieve. The default is the current date.	No
endDate	date	The end date in the date range of add-on acquisition data to retrieve. The default is the current date.	No
filter	string	One or more statements that filter the rows in the response. Each statement contains a field name from the response body and value that are associated with the eq or ne operators, and statements can be combined using and or or. String values must be surrounded by single quotes in the filter parameter. For example, filter=market eq 'US' and gender eq 'm'. You can specify the following fields from the response body: • acquisitionType • age • storeClient • gender • market • osVersion • deviceType • sandboxId	No
aggregationLevel	string	Specifies the time range for which to retrieve aggregate data. Can be one of the following strings: day , week , or month . If unspecified, the default is day .	No

Parameter	Type	Description	Required
orderby	string	A statement that orders the result data values for each add-on acquisition. The syntax is <i>orderby=field [order],field [order],...</i> The <i>field</i> parameter can be one of the following strings: • date • acquisitionType • age • storeClient • gender • market • osVersion • deviceType • orderName The order parameter is optional, and can be asc or desc to specify ascending or descending order for each field. The default is asc. Here is an example <i>orderby</i> string: <i>orderby=date,market</i>	No
groupby	string	A statement that applies data aggregation only to the specified fields. You can specify the following fields: • date • applicationName • addonProductName • acquisitionType • age • storeClient • gender • market • osVersion • deviceType • paymentInstrumentType • sandboxId • xboxTitleIdHex The returned data rows will contain the fields specified in the <i>groupby</i> parameter as well as the following: • date • applicationId • addonProductId • acquisitionQuantity The groupby parameter can be used with the <i>aggregationLevel</i> parameter. For example: <i>&groupby=age,market&aggregationLevel=week</i>	No

Request example

The following examples demonstrates several requests for getting add-on acquisition data. Replace the *addonProductId* and *applicationId* values with the appropriate Store ID for your add-on or app.

syntax

```
GET
https://manage.devcenter.microsoft.com/v1.0/my/analytics/addonacquisitions?
applicationId=9WZDNCRFJ314&startDate=1/1/2015&endDate=2/1/2015&top=10&skip=0
HTTP/1.1
```

Authorization: Bearer <your access token>

```
GET
https://manage.devcenter.microsoft.com/v1.0/my/analytics/addonacquisitions?
applicationId=9WZDNCRFJ314&startDate=1/1/2015&endDate=2/1/2015&top=10&skip=0
&filter=market eq 'GB' and gender eq 'm' HTTP/1.1
```

Authorization: Bearer <your access token>

Response

Response body

Value	Type	Description
Value	array	An array of objects that contain aggregate add-on acquisition data. For more information about the data in each object, see the add-on acquisition values section below.
TotalCount	int	The total number of rows in the data result for the query.

Add-on acquisition values

Elements in the Value array contain the following values.

Value	Type	Description
date	string	The first date in the date range for the acquisition data. If the request specified a single day, this value is that date.

Value	Type	Description
		If the request specified a week, month, or other date range, this value is the first date in that date range.
addonProductId	string	The <i>productId</i> of the add-on for which you are retrieving acquisition data.
addonProductName	string	The display name of the add-on. This value only appears in the response data if the <i>aggregationLevel</i> parameter is set to day , unless you specify the addonProductName field in the <i>groupby</i> parameter.
applicationId	string	The <i>productId</i> of the app for which you want to retrieve add-on acquisition data.
applicationName	string	The display name of the game.
deviceType	string	One of the following strings that specifies the type of device that completed the acquisition: • "PC" • "Phone" • "Console-Xbox One" • "Console-Xbox Series X" • "IoT" • "Server" • "Tablet" • "Holographic" • "Unknown"
storeClient	string	One of the following strings that indicates the version of the Store where the acquisition occurred: • "Windows Phone Store (client)" • "Microsoft Store (client)" (or "Windows Store (client)" if querying for data before March 23, 2018) • "Microsoft Store (web)" (or "Windows Store (web)" if querying for data before March 23, 2018) • "Volume purchase by organizations" • "Other"
osVersion	string	The OS version on which the acquisition occurred. For this method, this value is always either Windows 10 or Windows 11 .
market	string	The ISO 3166 country code of the market where the acquisition occurred.
gender	string	One of the following strings that specifies the gender of the user who made the acquisition: • "m"

Value	Type	Description
		• "f" • "Unknown"
age	string	One of the following strings that indicates the age group of the user who made the acquisition: • "less than 13" • "13-17" • "18-24" • "25-34" • "35-44" • "44-55" • "greater than 55" • "Unknown"
acquisitionType	string	One of the following strings that indicates the type of acquisition: • "Free" • "Trial" • "Paid" • "Promotional code" • "lap" • "Subscription lap" • "Private Audience" • "Pre Order" • "Xbox Game Pass" (or "Game Pass" if querying for data before March 23, 2018) • "Disk" • "Prepaid Code" • "Charged Pre Order" • "Cancelled Pre Order" • "Failed Pre Order"
acquisitionQuantity	integer	The number of acquisitions that occurred.
inAppProductId	string	Product ID of the product where this add-on is used.
inAppProductName	string	Product Name of the product where this add-on is used.
paymentInstrumentType	string	Payment instrument type used for the acquisition.
sandboxId	string	The Sandbox ID created for the game. This can be the value RETAIL or a private sandbox ID.
xboxTitleId	string	Xbox Title ID of the product from XDP, if applicable.
localCurrencyCode	string	Local Currency code based on the country of the Partner Center account.

Value	Type	Description
xboxProductId	string	Xbox Product ID of the product from XDP, if applicable.
availabilityId	string	Availability ID of the product from XDP, if applicable.
skuld	string	SKU ID of the product from XDP, if applicable.
skuDisplayName	string	SKU Display Name of the product from XDP, if applicable.
xboxParentProductId	string	Xbox Parent Product ID of the product from XDP, if applicable.
parentProductName	string	Parent Product Name of the product from XDP, if applicable.
productTypeName	string	Product Type Name of the product from XDP, if applicable.
purchaseTaxType	string	Purchase Tax Type of the product from XDP, if applicable.
purchasePriceUSDAmount	number	The amount paid by the customer for the add-on, converted to USD.
purchasePriceLocalAmount	number	The amount paid by the customer for the add-on, in the region's currency.
purchaseTaxUSDAmount	number	The tax amount applied to the add-on, converted to USD.
purchaseTaxLocalAmount	number	Purchase Tax Local Amount of the product from XDP, if applicable.

Response example

The following example demonstrates an example JSON response body for this request.

JSON

```
{
  "Value": [
    {
      "inAppProductId": "9NBLGGH1864K",
      "inAppProductName": "866879",
      "addonProductId": "9NBLGGH1864K",
      "addonProductName": "866879",
      "date": "2017-11-05",
      "applicationId": "9WZDNCRFJ314",
      "applicationName": "Tetris Blitz",
      "acquisitionType": "Iap",
      "age": "35-49",
      "deviceType": "Phone",
    }
  ]
}
```

```
        "gender": "m",
        "market": "US",
        "osVersion": "Windows Phone 8.1",
        "paymentInstrumentType": "Credit Card",
        "sandboxId": "RETAIL",
        "storeClient": "Windows Phone Store (client)",
        "xboxTitleId": "",
        "localCurrencyCode": "USD",
        "xboxProductId": "00000000-0000-0000-0000-000000000000",
        "availabilityId": "",
        "skuId": "",
        "skuDisplayName": "Full",
        "xboxParentProductId": "",
        "parentProductName": "Tetris Blitz",
        "productTypeName": "Add-On",
        "purchaseTaxType": "",
        "acquisitionQuantity": 1,
        "purchasePriceUSDAmount": 1.08,
        "purchasePriceLocalAmount": 0.09,
        "purchaseTaxUSDAmount": 1.08,
        "purchaseTaxLocalAmount": 0.09
    },
],

"@nextLink": null,

"TotalCount": 7601
}
```

Get app acquisitions

Article • 11/01/2022

Use this method in the Microsoft Store analytics API to get aggregate acquisition data in JSON format for an application during a given date range and other optional filters. This information is also available in the [Acquisitions report](#) in Partner Center.

Prerequisites

To use this method, you need to first do the following:

- If you have not done so already, complete all the [prerequisites](#) for the Microsoft Store analytics API.
- [Obtain an Azure AD access token](#) to use in the request header for this method. After you obtain an access token, you have 60 minutes to use it before it expires. After the token expires, you can obtain a new one.

Request

Request syntax

Method	Request URI
GET	<code>https://manage.devcenter.microsoft.com/v1.0/my/analytics/appacquisitions</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <i><token></i> .

Request parameters

Parameter	Type	Description	Required
applicationId	string	The Store ID of the app for which you want to retrieve acquisition data.	Yes

Parameter	Type	Description	Required
startDate	date	The start date in the date range of acquisition data to retrieve. The default is the current date.	No
endDate	date	The end date in the date range of acquisition data to retrieve. The default is the current date.	No
top	int	The number of rows of data to return in the request. The maximum value and the default value if not specified is 10000. If there are more rows in the query, the response body includes a next link that you can use to request the next page of data.	No
skip	int	The number of rows to skip in the query. Use this parameter to page through large data sets. For example, top=10000 and skip=0 retrieves the first 10000 rows of data, top=10000 and skip=10000 retrieves the next 10000 rows of data, and so on.	No
filter	string	One or more statements that filter the rows in the response. Each statement contains a field name from the response body and value that are associated with the eq or ne operators, and statements can be combined using and or or. String values must be surrounded by single quotes in the <i>filter</i> parameter. For example, <i>filter=market eq 'US' and gender eq 'm'</i>. You can specify the following fields from the response body: • acquisitionType • ageGroup • storeClient • gender • market • osVersion • deviceType • orderName	No
aggregationLevel	string	Specifies the time range for which to retrieve aggregate data. Can be one of the following strings: day , week , or month . If unspecified, the default is day .	No

Parameter	Type	Description	Required
orderby	string	A statement that orders the result data values for each acquisition. The syntax is <i>orderby=field [order],field [order],....</i> The <i>field</i> parameter can be one of the following strings: • date • acquisitionType • ageGroup • storeClient • gender • market • osVersion • deviceType • orderName The <i>order</i> parameter is optional, and can be asc or desc to specify ascending or descending order for each field. The default is asc. Here is an example <i>orderby</i> string: <i>orderby=date,market</i>	No
groupby	string	A statement that applies data aggregation only to the specified fields. You can specify the following fields: • date • applicationName • acquisitionType • ageGroup • storeClient • gender • market • osVersion • deviceType • orderName The returned data rows will contain the fields specified in the <i>groupby</i> parameter as well as the following: • date • applicationId • acquisitionQuantity The <i>groupby</i> parameter can be used with the <i>aggregationLevel</i> parameter. For example: <i>&groupby=ageGroup,market&aggregationLevel=week</i>	No

Request example